

NUME: PĂCURAR IRINA
UTCN, CTI
Grupa 30644



EchoRO

Mobile App



MARTIE 2026

Cuprins

1. Descrierea aplicației.....	2
2. Arhitectura conceptuală	2
2.1. Modulul de Prezentare (Frontend - Aplicația Mobilă).....	2
2.2. Modulul de logică (Backend & Procesare AI)	2
2.3. Modulul de date (Baza de date).....	3
2.4. Comunicarea între module.....	3
3. Figma Design	4
4. Arhitectura tehnică.....	7
4.1. Nivelul Client (Frontend Mobil).....	7
4.2. Nivelul Server (Backend & Inteligență Artificială)	7
4.3. Nivelul de Persistență (Baza de Date)	8
4.4. Protocoale de Comunicare și Formate de Date.....	8
5. Modelul de date – diagrama	8
6. Interfețe de comunicare.....	8

1. Descrierea aplicației

Aplicația mobilă **EchoRO** este o soluție software dedicată platformei Android, având ca obiectiv conversia textului în vorbire (Text-to-Speech) de înaltă fidelitate pentru limba română. Sistemul se bazează pe o arhitectură client-server: interfața mobilă preia textul și preferințele de stilizare vocală ale utilizatorului, în timp ce procesarea complexă este realizată de un server (Backend) care rulează motorul de inteligență artificială Parler-TTS.

Platforma integrează două niveluri de acces. Modul **Guest** (fără cont) permite utilizarea unui model AI "Mini", optimizat pentru viteză și generări scurte. În schimb, utilizatorii autentificați în modul **Pro** au acces la modelul "Large", care oferă o intonație mult mai naturală și o pronunție precisă.

Pentru a susține componenta de cercetare a proiectului, EchoRO include un **modul de feedback** prin care utilizatorii pot evalua calitatea vocii generate folosind metricile standardizate MOS (inteligibilitate, naturalitate, accent și acuratețe). Aceste date sunt centralizate și pot fi vizualizate prin intermediul unui **Tablou de Bord pentru Administrator**, permițând monitorizarea și compararea performanțelor celor două modele AI în timp real.

2. Arhitectura conceptuală

Platforma EchoRO este proiectată pe baza unei arhitecturi clasice pe 3 niveluri (3-tier architecture), care separă clar interfața de utilizator de logica de procesare și de stocarea datelor. Acest tip de decuplare asigură o mentenanță ușoară, scalabilitate și o performanță optimă pentru dispozitivele mobile. Sistemul este alcătuit din următoarele 3 module principale:

2.1. Modulul de Prezentare (Frontend - Aplicația Mobilă)

Acest modul reprezintă aplicația mobilă nativă, dezvoltată în Kotlin pentru sistemul de operare Android. Este singura componentă cu care utilizatorul interacționează direct.

- *Funcționalități:* Gestionează interfața grafică (UI), preia textul și setările de voce de la utilizator (prin tastare sau dictare vocală), redă fișierele audio generate (.wav) și expune formularele de evaluare (MOS și procentajul de acuratețe).
- *Rol arhitectural:* Acționează ca un client care consumă resursele expuse de server, fără a executa procesări grele la nivel local.

2.2. Modulul de logică (Backend & Procesare AI)

Acest modul reprezintă creierul platformei, dezvoltat în Python (utilizând framework-ul FastAPI). El funcționează ca un intermediar între aplicația mobilă și baza de date, dar integrează și motorul de inteligență artificială.

- *Gestiunea API-ului și a Autentificării:* Expune endpoint-urile de rețea (REST) și verifică permisiunile utilizatorilor (oferind acces la modelul Mini pentru oaspeți și la modelul Large pentru cei autentificați).

- *Motorul de Inferență (Parler-TTS)*: În cadrul acestui modul este încărcat modelul de Deep Learning. Backend-ul preia textul, pregătește prompturile și rulează procesul de sinteză vocală (Text-to-Speech) utilizând resursele hardware ale serverului (GPU), generând fișierul audio final.

2.3. Modulul de date (Baza de date)

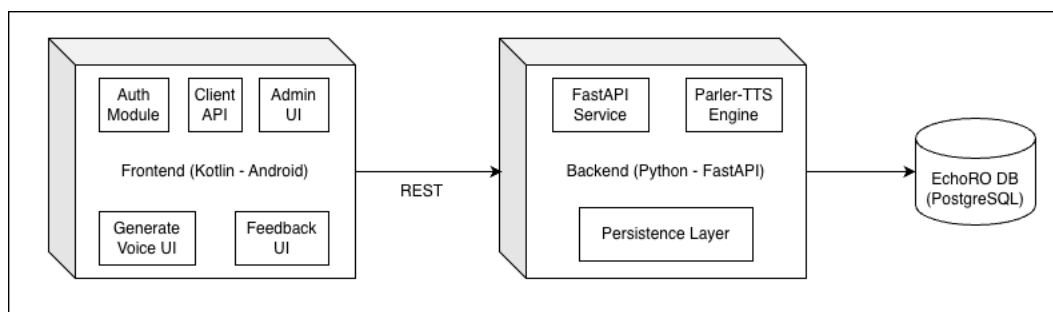
Acest nivel este responsabil pentru persistența datelor, asigurând stocarea, recuperarea și actualizarea informațiilor în mod securizat.

- *Rol arhitectural*: Funcționează complet independent de interfață, comunicând exclusiv cu modulul Backend.
- *Date stocate*: Păstrează credențialele utilizatorilor autentificați, istoricul cererilor de generare (textul introdus și calea către fișierul audio) și datele analitice colectate din formularele de feedback (notele de evaluare a performanței modelului).

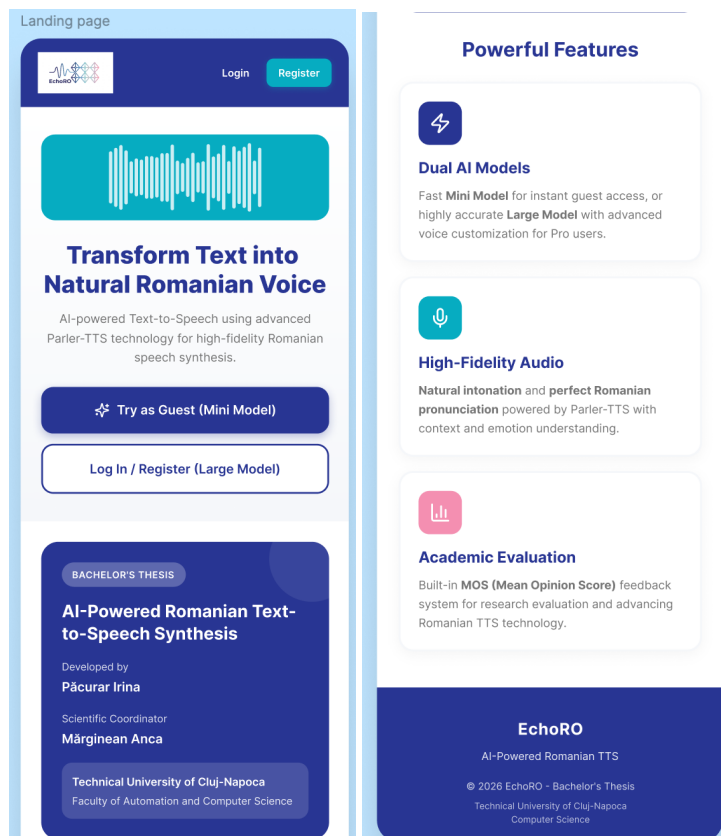
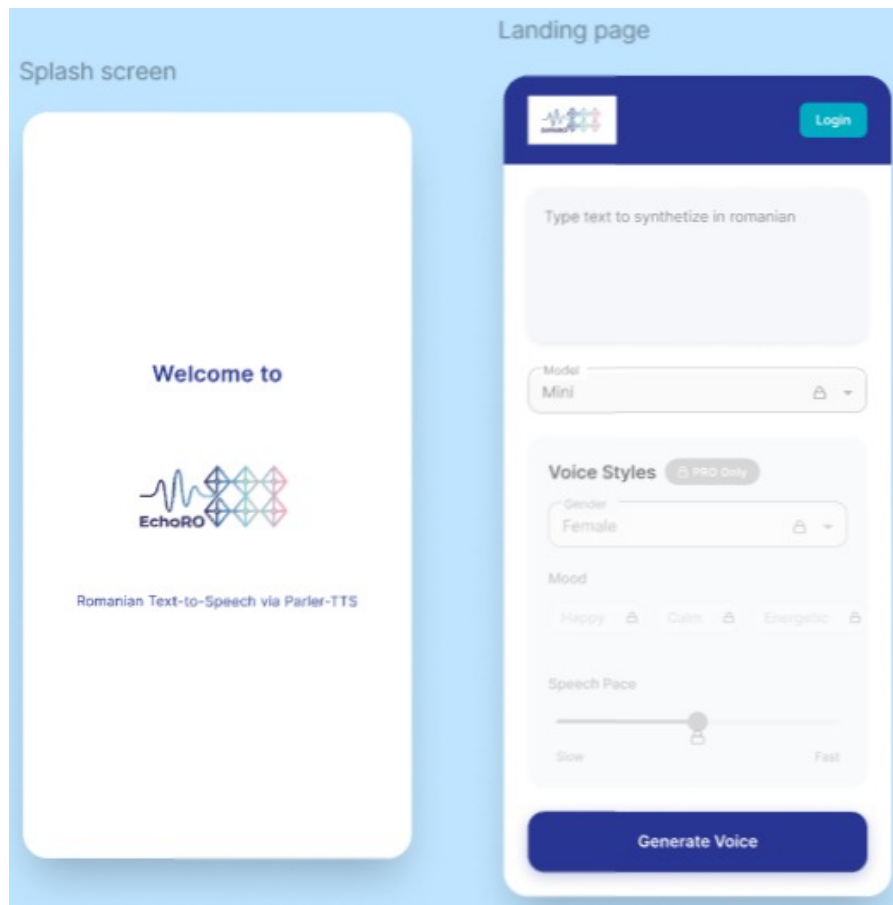
2.4. Comunicarea între module

Comunicarea între cele trei module se realizează secvențial și securizat, după cum urmează:

1. *Frontend → Backend (Cererea)*: Utilizatorul introduce textul în aplicația mobilă. Modulul Frontend serializează datele în format JSON și inițiază un request HTTP (tip POST) către API-ul expus de Modulul Backend.
2. *Backend ↔ Baza de Date (Validarea)*: Backend-ul primește cererea, interoghează Baza de Date pentru a valida sesiunea utilizatorului și a determina nivelul de acces la modelele AI.
3. *Procesarea internă (Backend)*: Modulul Backend rulează modelul Parler-TTS pe baza datelor primite și generează fișierul audio.
4. *Backend → Baza de Date (Salvarea)*: Odată generată vocea, Backend-ul înregistrează în Baza de Date istoricul tranzacției.
5. *Backend → Frontend (Răspunsul)*: Backend-ul returnează aplicației mobile fișierul audio generat, pe care Frontend-ul îl redă utilizatorului. Ulterior, același ciclu de comunicare este reluat pentru trimiterea și stocarea formularului de feedback.



3. Figma Design



Login page


EchoRO Pro

[Create a New Account](#)

LOGIN TO PRO MODEL

Register page


EchoRO Pro

CREATE PRO ACCOUNT

[Already have an account? Login.](#)

[Continue as Guest](#)

Generate sound page



Model

Mini

Voice Styles

Gender

Female

Mood

Happy

Calm

Energetic

Profes

Speech Pace

Slow

Fast

Generate Voice

Loading page




Generating audio with trained Parler-TTS model

[Estimated time: 5-15 seconds](#)

Feedback page







Detailed Feedback (including MOS)

1. Intelligibility

☆☆☆☆☆

2. Naturalness

☆☆☆☆☆

3. Accent

☆☆☆☆☆

Percentage of words pronounced correctly

Slide to indicate the percentage of correctly pronounced words


0% 100%

85%

Optional comments...

Submit Feedback

AdminView



4. Arhitectura tehnică

Platforma EchoRO utilizează o stivă tehnologică personalizată, decuplată de infrastructurile web tradiționale (precum LAMP sau MERN). Această abatere asumată este strict necesară și justificată de specificul temei de licență: integrarea și rularea nativă a unor modele complexe de inteligență artificială (Parler-TTS și modele Speech-to-Text), care impun utilizarea ecosistemului Python la nivel de backend și necesită accelerare hardware (GPU).

Arhitectura este de tip Client-Server, utilizând următoarele tehnologii și protocoale:

4.1. Nivelul Client (Frontend Mobil)

Pentru dezvoltarea aplicației mobile s-a optat pentru dezvoltarea nativă pe platforma Android.

- *Kotlin*: limbajul oficial și recomandat de Google pentru dezvoltarea Android. Oferă siguranță sporită (null-safety), o sintaxă concisă și performanță nativă, esențială pentru o interfață fluidă.
- *Retrofit*: bibliotecă pentru gestionarea cererilor de rețea. Permite maparea ușoară a endpoint-urilor RESTful direct în interfețe Kotlin, serializând automat datele și gestionând eficient apelurile asincrone către server.
- *ExoPlayer (Media3)*: Bibliotecă pentru redarea media. Oferă un control mult mai granular și stabil asupra fluxurilor audio comparativ cu MediaPlayer-ul standard din Android, fiind capabil să gestioneze fișierele *.wav* returnate de modelul TTS.

4.2. Nivelul Server (Backend & Inteligență Artificială)

Backend-ul unifică logica de business cu motorul de inferență AI într-un singur serviciu monolit, pentru a minimiza latența.

- *Python*: Limbajul de bază al serverului. Este standardul absolut în industria de Machine Learning. Orice tentativă de a folosi Java sau Node.js pe backend ar fi necesitat un proces separat pentru Python, introducând întârzieri masive în comunicarea inter-procese.
- *FastAPI*: Framework web pentru construirea API-urilor în Python. Este unul dintre cele mai rapide framework-uri Python, suportând operații asincrone (AsyncIO) în mod nativ. Permite menținerea modelului de inteligență artificială încărcat în memoria RAM/VRAM, răspunzând instantaneu la noile cereri HTTP fără a reîncărca modelul la fiecare request.
- *PyTorch & Hugging Face Transformers*: Bibliotecile de Deep Learning. Necesare pentru instanțierea, optimizarea (în format float16) și rularea modelelor Parler-TTS (Mini și Large) pe procesorul grafic dedicat (NVIDIA RTX), precum și pentru integrarea funcționalității Speech-to-Text (ex. modelul Whisper).

4.3. Nivelul de Persistență (Baza de Date)

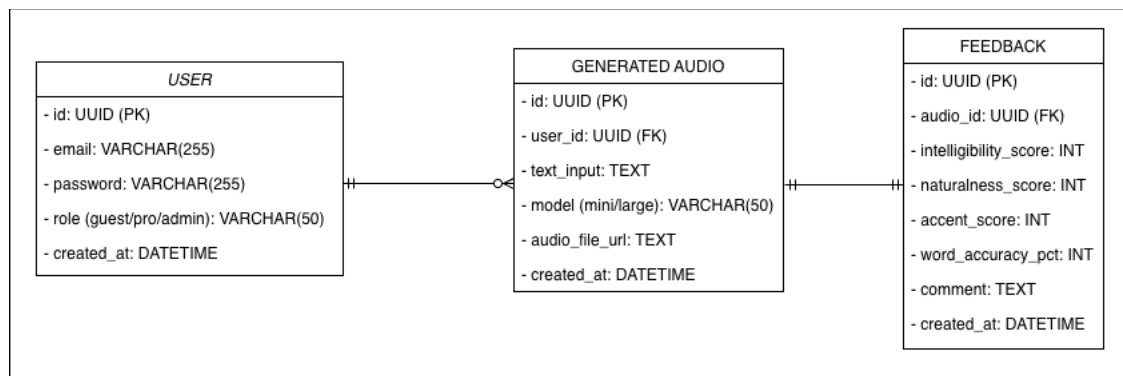
- *PostgreSQL*: Sistem de gestiune a bazelor de date relaționale (RDBMS). Oferă integritate referențială strictă, securitate robustă și scalabilitate superioară. Este ideal pentru a stoca entități cu relații clare: utilizatori, roluri (Guest/Pro), istoricul cererilor de generare și rezultatele formularelor de evaluare (metricile MOS).

4.4. Protocoale de Comunicare și Formate de Date

Pentru a asigura un flux de date securizat și standardizat între aplicația mobilă (Kotlin) și serverul AI (Python), au fost alese următoarele standarde:

- *HTTP/HTTPS (REST API)*: Arhitectura de tip REST a fost aleasă pentru simplitatea și standardizarea sa. Comunicarea se realizează exclusiv prin metode HTTP specifice (*POST* pentru trimiterea textului și a setărilor, *GET* pentru descărcarea fișierelor audio și obținerea statisticilor).
- *JSON (JavaScript Object Notation)*: Utilizat ca format standard de serializare a datelor pentru corpul cererilor și răspunsurilor (de exemplu, transmiterea parametrilor de gen, stare de spirit și text).
- *Transfer binar (Media Streaming)*: Pentru livrarea rezultatului, serverul răspunde cererilor valide returnând direct fluxul de octeți al fișierului audio (*audio/wav*), aplicația mobilă decodând acest stream în timp real.

5. Modelul de date – diagrama



6. Interfețe de comunicare

Metoda	Endpoint	Payload/Response	Descriere
POST	/login	{ "email": "String", "password": "String" }	Autentifică utilizatorul
POST	/register	{ "full_name": "String", "email": "String", "password": "String", }	Creează un cont nou

		"confirm_password": "String" }	
POST	/generate	Payload: { "text": "String", "model": "String", "voice_settings": { "gender": "String", "mood": "String", "pace": "Float" } } Response: { "generation_id": "String", "audio_url": "String", "processing_time_seconds": "Float" } }	Generează fișierul audio pe baza textului
POST	/feedback	{ "generation_id": "String", "metrics": { "mos_intelligibility": "Integer", "mos_naturalness": "Integer", "mos_accent": "Integer", "word_accuracy_percentage": "Integer", }, "optional_comments": "String" } }	Salvează rezultatele evaluării modelului (MOS)
GET	/stats	{ "total_generations": "Integer", "overall_mos": "Float", "model_comparison": { "large_model": { "avg_intelligibility": "Float", "avg_naturalness": "Float", "avg_accent": "Float", "avg_word_accuracy": "Float" }, "mini_model": { "avg_intelligibility": "Float", "avg_naturalness": "Float", "avg_accent": "Float", "avg_word_accuracy": "Float" } } } }	Returnează statisticile agregate pentru modele